

快手直播间AI语音导购Agent — 项目介绍

快手AI产品经理岗位·校招面试作品集 作者: wsd26 | 日期: 2026年5月

一、项目概述

1.1 产品定位

快手直播间AI语音导购Agent "小快"

是一个模拟快手直播购物场景的AI产品Demo。用户可以在模拟直播间中通过**纯语音对话**与AI导购交互，完成商品咨询、智能推荐、语音加购、自动监控等完整购物决策链路。

核心差异化在于：**不是通用聊天机器人，而是深度嵌入快手直播间上下文的专业购物Agent** — 它知道当前主播在展示什么、记得用户偏好、能跨商品对比、会主动观察推荐。

1.2 目标用户与痛点

用户类型	痛点	AI导购价值
直播购物新手	节奏快、不敢提问、怕买错	1v1私密解答，消除决策焦虑
通勤/忙碌用户	无法打字，需要免手操作	纯语音交互，边看直播边问
比价型用户	多个商品难对比	记住上下文，跨商品对比
复购用户	想更快决策	记住偏好，一键推荐

1.3 核心场景

- 商品咨询**："这个衣服什么材质？" → AI基于当前商品信息回答
- 个性化推荐**："我黄黑皮适合哪个颜色？" → AI结合用户画像推荐
- 商品对比**："这个面膜和刚才那个哪个划算？" → AI跨时间对比
- 尺码建议**："170cm穿M还是L？" → AI查阅尺码表+用户身材数据
- 语音加购**："帮我加购物车" → 语音触发加购动作
- 主动监控**：新品上架时AI自动评分，高分商品主动推送给用户

二、产品设计

2.1 用户交互流程

进入直播间 → Onboarding偏好设置(4步引导)
→ AI自动语音问候 → 用户按住说话提问
→ AI思考(头像脉冲动画) → AI语音回复 + 文字气泡 + 商品推荐卡片
→ 快捷操作入口(加购物车/详细/对比)
→ 继续对话 or 下单决策

2.2 AI导购面板三态设计

状态	表现	使用场景
折叠态	右下角圆形头像按钮，不遮挡直播画面	用户专注看直播
半屏态	底部弹出，显示最近3-4条对话+录音按钮	快速问答
全屏态	覆盖60%屏幕，完整对话历史+可滚动	深度咨询、商品对比

2.3 核心交互约束

- **Push-to-Talk**：按住说话、松开发送
- **Barge

2.4 状态处理矩阵

状态	组件表现
AI思考中	头像脉冲动画，气泡LoadingDots，录音键禁用
对话为空	AI主动问候，提示用户可以问什么
网络错误	ErrorBanner + 重试按钮
麦克风权限拒绝	文字输入降级
未识别到语音	Toast"没有听清，请再说一次"
浏览器不支持语音	降级为文字输入模式
录音超时(10s)	自动结束+Toast提示
购物车为空	引导文案+"让小快帮你挑选"
用户打断TTS	立即停止播放，开始新录音
用户上滑查看历史	不自动滚动，显示"新消息↓"箭头

三、商业模式与竞争壁垒（面试论述重点）

3.1 GMV驱动

每次AI解答都是一次成交机会。消除决策犹豫 = 提升转化率。AI可以：

- 即时解答疑虑（材质、尺码、价格）→ 减少"犹豫跳出"
- 个性化推

3.2 用户增长

- **降低新人门槛**：语音交互比打字更自然，覆盖不擅长打字的用户
- **无障碍

3.3 数据飞轮

用户交互 → 画像沉淀(偏好/预算/肤色/尺码)
→ 推荐更精准 → 转化率提升
→ 用户粘性增强 → 更多交互数据
→ 画像更精准 → 飞轮加速

3.4 竞争壁垒

- **直播间上下文感知**：AI知道当前主播在展示什么、刚才展示过什么
- **平台资源

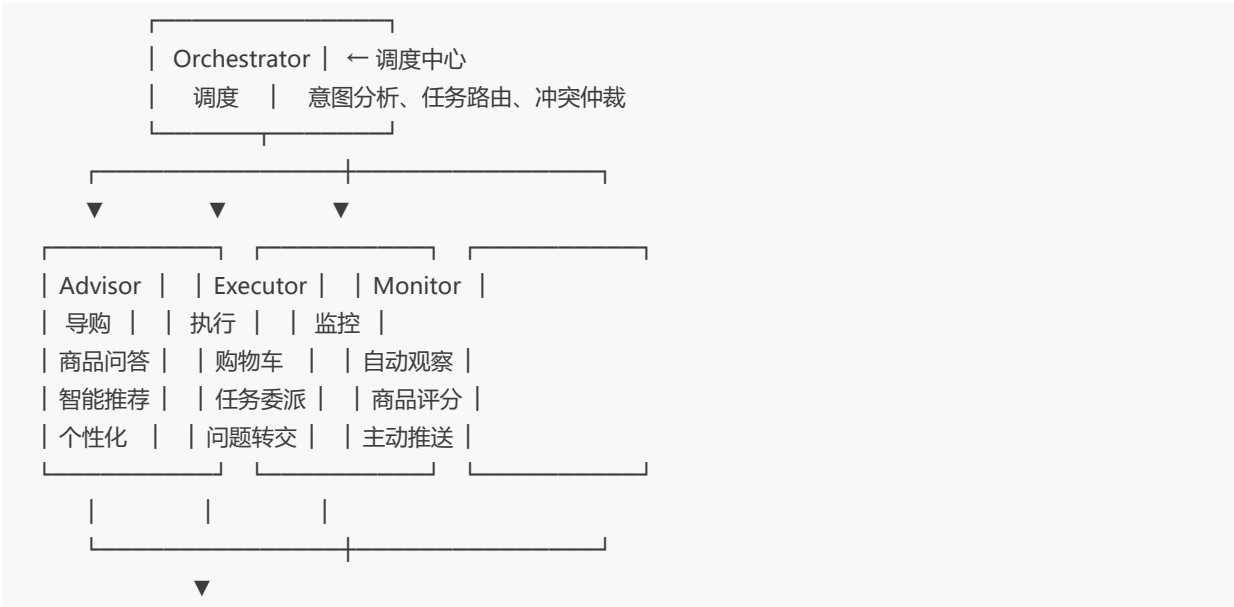
四、技术架构

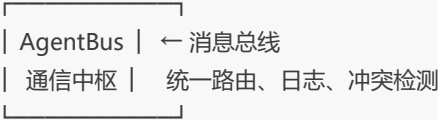
4.1 技术栈

层	技术	选择理由
构建工具	Vite 8 + React 19 + TypeScript	现代前端标配，HMR极快
样式	Tailwind CSS 3 + Framer Motion	快速开发+流畅动画
状态管理	Zustand 5	轻量、支持持久化、无Provider
语音STT	Web Speech API (SpeechRecognition)	零依赖、Chrome原生支持中文
语音TTS	Web Speech API (SpeechSynthesis)	零成本、中文语音自然
路由	React Router v7 (HashRouter)	SPA路由，兼容GitHub Pages
部署	GitHub Pages + GitHub Actions	免费、可分享链接
测试	Vitest	与Vite生态统一

4.2 多Agent架构

项目采用**4-Agent协作架构**，模拟真实AI产品中多智能体协同工作的模式：





Agent职责分工

Agent	类型	职责
OrchestratorAgent (调度中心)	orchestrator	意图分析引擎、任务路由、冲突仲裁
AdvisorAgent (导购Agent)	advisor	商品问答、智能搜索、个性化推荐
ExecutorAgent (执行Agent)	executor	加购操作、任务委派、问题转交主播
MonitorAgent (监控Agent)	monitor	自动观察商品、偏好评分、主动推送

消息协议

所有Agent通过`AgentBus`以统一消息格式通信：

```
消息格式: { from, to, type, payload, priority, id, timestamp }
消息类型: user_input | product_query | product_search | command_request
          | monitor_alert | agent_response | status_update | conflict_resolve
```

通信流程示例: "裙子包邮吗"

1. UI → Orchestrator [user_input: "裙子包邮吗"]
2. Orchestrator 意图分析: "裙子"→category clothing + "包邮"→hasQuestion
→ 找到最佳匹配商品(连衣裙) → type: specific_product
3. Orchestrator → Advisor [product_query: product=连衣裙, text="裙子包邮吗"]
4. Advisor 处理: 匹配"包邮"正则 → 返回包邮回复 + 商品卡片
5. Advisor → UI [agent_response]
6. UI 更新对话 + TTS语音播放

4.3 意图分析引擎

OrchestratorAgent内置基于正则的意图分析引擎，支持：

- **问候检测**： 你好/hi/在吗/主播好...

- **命令格

4.4 MonitorAgent评分引擎

MonitorAgent持续观察直播间商品，对每个新品计算匹配分：

```
基础分: 50
+ 预算匹配: +15 (范围内) / +5 (低于) / +5 (略超≤30%)
+ 品类偏好: +20 (匹配用户偏好品类)
+ 肤色匹配: +10 (clothing/skincare品类)
+ 好评率: +8 (rating ≥ 4.7)
+ 销量: +7 (sales > 5000)
```

+ 折扣深度: +10 (price/original ≤ 50%)

推送阈值: ≥ 65分 → 主动推荐

4.5 Zustand Store 状态管理

Store	职责	持久化
`useConversationStore`	消息列表、录音/思考/说话状态、Agent模式、面板状态	内存
`useLiveStreamStore`	当前商品、主播信息、弹幕、商品切换	内存
`useCartStore`	购物车增删、抽屉开关、总金额	内存
`useUserStore`	偏好设置(肤色/预算/品类/关注点)、onboarding状态	localStorage

4.6 目录结构

```
ai-voice-shopping-guide/
├── src/
│   ├── main.tsx           # 应用入口
│   ├── App.tsx           # 路由配置
│   ├── index.css         # Tailwind + 全局样式
│   ├── types/            # TypeScript类型定义
│   │   ├── index.ts      # 导出入口
│   │   ├── product.ts    # Product, CartItem
│   │   ├── conversation.ts # Message, ChatResponse, ProductCardData
│   │   └── user.ts       # UserPreferences
│   ├── constants/        # 静态数据
│   │   ├── products.ts   # 8个Mock商品(服装/护肤/配饰/数码/食品)
│   │   └── liveStream.ts # 直播配置、弹幕生成
│   ├── store/            # Zustand状态管理
│   │   ├── useConversationStore.ts # 对话状态
│   │   ├── useLiveStreamStore.ts  # 直播状态
│   │   ├── useCartStore.ts        # 购物车状态
│   │   └── useUserStore.ts        # 用户画像(localStorage持久化)
│   ├── agents/           # 多Agent系统
│   │   ├── types.ts      # Agent接口/消息协议定义
│   │   ├── AgentBus.ts   # 通信总线(路由/日志/冲突检测)
│   │   ├── OrchestratorAgent.ts # 调度中心(意图分析+路由)
│   │   ├── AdvisorAgent.ts # 导购Agent(商品问答+推荐)
│   │   ├── ExecutorAgent.ts # 执行Agent(加购+任务委派)
│   │   ├── MonitorAgent.ts  # 监控Agent(观察+评分+推送)
│   │   └── index.ts       # 初始化注册
│   ├── hooks/            # 自定义Hooks
│   │   ├── useAIShoppingGuide.ts # AI导购全流程串联
│   │   ├── useSpeechRecognition.ts # 语音识别(STT)
│   │   ├── useSpeechSynthesis.ts  # 语音合成(TTS)
│   │   └── useAutoScroll.ts      # 消息列表自动滚动
│   ├── components/       # 组件
│   │   ├── AIGuide/      # AI导购面板(头像/气泡/波形/推荐卡片)
│   │   ├── LiveStream/   # 直播画面(主播/商品/弹幕/点赞)
│   │   ├── Cart/         # 购物车(抽屉/列表)
│   │   ├── Product/      # 商品(详情/对比)
│   │   └── Layout/       # 布局(MobileFrame手机模拟框)
```

```
| | └─ Common/          # 通用(Toast/Error/Empty/Loading)
| └─ pages/
|   └─ LandingPage.tsx   # Onboarding偏好引导(4步)
|   └─ LiveRoomPage.tsx  # 直播间主页
└─ .github/workflows/deploy.yml # CI/CD自动部署
└─ package.json
└─ vite.config.ts
└─ tailwind.config.js
└─ README.md
```

五、核心功能实现

5.1 语音交互全链路

用户按住录音按钮

- Web Speech API (SpeechRecognition) 实时转文字
- AgentBus.dispatch(user_input) 发送给Orchestrator
- Orchestrator分析意图 → 路由到Advisor/Executor
- Agent处理 → 返回ChatResponse
- UI更新对话气泡 + 商品推荐卡片
- Web Speech API (SpeechSynthesis) 播报回复
- 播放完毕, 按钮恢复"按住说话"

5.2 Barge-in打断机制

采用**Generation Counter**模式解决异步回调竞态问题:

1. 用户按下按钮 → `stop()`取消TTS + `generationRef += 1` + 开始录音
2. 新录音识别完成 → 生成新的generation → `speakNext(generation)` 播放
3. 如果旧generation的回调触发 → `generation !== generationRef.current` → 直接return

5.3 智能意图路由规则

用户输入	意图类型	路由目标	处理
"你好"/"在吗"	greeting	Advisor	问候+引导
"裙子包邮吗"	specific_product	Advisor	找最佳匹配→检查包邮
"有手表吗"	product_search	Advisor	搜索→未找到→诚实回复+转交主播
"有裙子吗"	specific_product	Advisor	精确无匹配→分类回退→找连衣裙
"这个多少钱"	current_product	Advisor	基于当前商品回答
"加入购物车"	command	Executor	加购操作
"帮我盯着"	command	Executor	开启监控模式
"面膜"	product_search	Advisor	分类搜索推荐

5.4 Mock商品库

8个模拟商品覆盖5个品类:

ID	名称	品类	价格	评分	销量
p001	春季新款法式碎花连衣裙	clothing	¥129	4.8	2341
p002	玻尿酸补水保湿面膜 10片装	skincare	¥49.9	4.9	15234
p003	复古牛皮手工小方包	accessories	¥358	4.7	892
p004	美白淡斑精华液 30ml	skincare	¥189	4.6	6721
p005	网红同款透气飞织运动鞋	clothing	¥199	4.5	4521
p006	新疆和田大枣 500g×2袋	food	¥39.9	4.9	32156
p007	简约纯棉宽松T恤 多色可选	clothing	¥69	4.7	8972
p008	智能蓝牙降噪耳机 Pro版	electronics	¥299	4.4	3412

六、产品数据指标体系（面试可论述）

6.1 北极星指标

AI辅助决策率：有AI交互的观看中，最终下单的比例

6.2 过程指标

指标	定义	目标
AI触发率	使用AI导购的用户占比	> 15%
对话完成率	用户提问后AI成功回复的比例	> 90%
语音识别准确率	STT正确识别中文的比例	> 95%
主动推荐CTR	Monitor推送的推荐卡片点击率	> 20%
加购转化率	AI交互→加购的比例	> 8%
用户满意度	反馈比例	> 85%
问题转交率	需转交主播的问题占比	< 10%

6.3 A/B实验设计

- **实验组**：**直播间启用AI语音导购

- **对照组**：**

七、技术亮点

7.1 多Agent协作架构

不是单体LLM调用，而是4个独立Agent通过消息总线协作。每个Agent有明确的职责边界、状态管理和通信协议。这模拟了真实生产环境中Multi-Agent System的设计模式。

7.2 零依赖语音方案

使用浏览器原生Web Speech

API实现STT+TTS，无需任何第三方语音服务。中文识别准确率在Chrome上有保障，且完全免费。

7.3 离线Mock数据闭环

整个Demo完全自包含 —— 不需要后端、不需要API

Key、不需要数据库。商品数据、直播模拟、对话管理全部在浏览器端完成。这使得项目可以部署为纯静态站点（GitHub Pages）。

7.4 Generation Counter异步控制

通过递增计数器模式解决TTS打断场景中的异步回调竞态，确保旧的播放回调不会污染新的播放状态。

7.5 正则意图引擎

在Demo场景中，用精心设计的正则规则引擎替代LLM做意图分析，实现零延迟、零成本、结果确定性强的意图路由。

八、未来迭代方向

8.1 接入真实LLM

当前意图分析使用规则引擎，未来可接入大模型（如DeepSeek）实现：

- 更灵活的意图理解

- 多轮对话

8.2 主播端联动

- AI无法回答时自动推送到主播端"问题看板"

- 主播可以

8.3 用户画像深化

- 基于浏览历史+购买记录的冷启动画像

- 实时更新

8.4 多模态交互

- 拍照搜同款（用户上传图片找直播间相似商品）

- AI虚拟开

九、部署与访问

- **源码仓库**: <https://github.com/wsd26/ai-shopping-agent>

• **在线D**

十、项目总结

本项目从**产品思维**（PRD/用户痛点/商业价值/数据指标）、**交互设计**（三态面板/Push-to-Talk/打断机制/状态矩阵）、**技术实现**（多Agent架构/语音管线/意图引擎/状态管理）三个维度，完整呈现了一个AI产品从0到1的设计与开发过程。

核心体现的能力：

1. **产品sense**：从用户痛点出发设计功能，而非从技术出发堆砌能力
2. **快手业务理解**：深度结合直播场景，考虑主播-用户-AI三方关系
3. **AI技术落地**：将Multi-Agent架构应用于真实产品场景
4. **工程实现能力**：完整的前端项目，可部署可体验可测试
5. **数据驱动思维**：设计指标体系，考虑A/B实验和ROI评估